지능정보시스템 중간고사 정리

인천대 컴퓨터공학부 · 2~6주차 + 10/13, 10/15 복습수업 반영 · 정리 기준: 수업 슬라이드, 수업 녹음

시험 정보

- 10/22(목) 오전 9시, 75분. 객관식 + 주관식. 주관식은 외우기보다 인과관계(왜 그런지)를 서술하는 형태.
- 코드 빈칸 채우기 없음 (코드는 구조만 이해). nDCG·cron·bitsandbytes는 출제 안 함.
- "이럴 때는 이걸 고른다" 형태(상황 → 선택)와 "이 증상은 어느 계층 원인이고 어떻게 해결하나"를 묻는다.
- 교수님이 직접 해보라고 한 것: ① BPE 손계산 ② documents CRUD를 `메서드 + URI` 표로 설계 ③ 학습 단계별 출력 차이 ④ 증상 → 계층 → 해결 매칭 ⑤ 체인의 구성요소와 입출력 형태 ⑥ 검색 품질 기법을 상황별로 고르기

2주차 · LLM 기초와 AI 시스템 개요

LLM 정의

- 방대한 텍스트로 학습해 다음 토큰을 확률로 예측하는 모델. 이 예측을 반복해 문장 생성 = 자기회귀(autoregressive).
- 요약·번역·분류·코드 생성이 모두 "다음 단어 예측"으로 표현됨 → 한 모델이 범용 도구가 됨.

종류	한 줄 정리	생긴 이유
VLM	이미지 + 텍스트를 같은 임베딩 공간에서 함께 입력	사진·표·화면 이해
MLLM	이미지에 음성·영상까지 확장	다양한 입력
SLM	수억~수십억 파라미터 경량 모델	GPU가 부족한 기기(노트북·폰)
LRM	답 전에 추론 과정을 길게 생성(느리고 비쌈, 정확)	초기 LLM엔 추론 기능이 없어 추론 과정을 학습시킴
MoE	전체 파라미터는 크지만 입력마다 일부만 활성화	작은 GPU로도 좋은 성능
도메인 특화	코드·의료·법률 데이터로 추가 학습	특정 분야 성능

Transformer와 Attention

- 이전 RNN/LSTM: 순차 처리라 병렬화 불가, 문장이 길면 앞부분을 잊음. Transformer(2017): 한 번에 전체를 봄. 핵심 = Attention.
- Q(질문 토큰), K(각 토큰의 표지), V(실제 값). 과정: Q·K 내적(유사도) → 값이 튀지 않게 나눔(스케일링) → softmax(0~1 확률) → V를 그 비율로 섞음.
- 멀티헤드: 같은 Q를 여러 관점(가리키는 대상 / 서술어 / 주어 / 속한 절)으로 동시에 계산. 위치 인코딩: 어텐션은 순서를 모르므로 필요. 요즘 LLM은 디코더 전용.

> 단점: 계산량 $O(n^2)$ — 모든 토큰이 모든 토큰과 비교. 토큰 2배 → 계산 4배. → 컨텍스트 윈도우에 상한, 길수록 비용·지연 급증 → "일단 다 넣자"가 안 통하고, 무엇을 넣을지 고르는 것이 곧 성능이자 비용 (= RAG·컨텍스트 엔지니어링이 필요한 이유).

토큰화 · BPE

- 토큰 = 모델의 최소 단위, 서브워드. 단어 단위는 신조어·오타 처리 불가, 글자 단위는 시퀀스가 너무 길어짐 → 서브워드가 절충안.
- BPE: 자주 함께 등장하는 문자쌍을 반복 병합, 정해 둔 어휘 크기에 도달할 때까지. 병합할 때마다 어휘가 1개씩 늘어남. 예) low×5, lower×2, newest×6, widest×3 → 초기 글자 10개 → `es`(9회) 병합 → 11개 → `est` → 12개 …
- 학습 순서: 말뭉치 준비 → 일부로 BPE 학습(어휘·병합 규칙 확정) → 그 토크나이저로 전체 말뭉치를 번호로 변환 → 그 번호로 LLM 사전학습.
- 로컬 모델은 그 모델이 학습한 토크나이저를 그대로 써야 함(다르면 추론 품질 저하). 모델 다운로드 시 함께 들어 있음.
- 한국어가 토큰을 더 쓰는 이유: 영어 데이터 비중이 커서 한국어 병합 기회가 적음 + 조사·어미 변화로 같은 어간이 여러 형태("학교에서/를/가")로 흩어져 빈도 분산 + byte-level BPE에서 한글 1글자 = 3바이트. 한 단어도 토큰 수가 다를 수 있음.

- 컨텍스트 윈도우 = 한 번에 입력할 수 있는 전체 토큰 길이(시스템 지시 + 도구 명세 + 이력 + 검색 결과 + 질문 합산). 요금 = 입력 + 출력 토큰(출력이 더 비쌈). 대화는 매번 이력 재전송 → 누적 비용은 턴 수의 제곱에 비례.

학습 3단계 (입력 "대한민국의 수도는" 기준)

단계	하는 일	결과
사전학습	인터넷급 대규모 텍스트로 다음 토큰 예측(정답 라벨 없음 = 자기 지도). 수 PB~15T 토큰	이어서 쓰기만 함: "서울이다. 인구는…"
SFT(지도 미세조정)	좋은 질문-답변 쌍 모방. 새 지식이 아니라 응답 형식을 가르침	"서울입니다."
정렬(RLHF/DPO)	두 답 중 나은 쪽을 학습(강화학습 개념). 도움이 되는 답 / 위험 요청 거절	톤·형식 일관, 위험 요청 거절, 모호하면 되묻기

학습 가능성: 사전학습 불가 · 전체 미세조정 사실상 어려움 · LoRA/QLoRA는 GPU 1~2장으로 제한적 가능 · RAG·프롬프트는 학습 불필요(즉시 가능).

양자화

- 가중치(웨이트 파일 속 실수들)의 숫자 표현 비트를 줄여 메모리 절감. 성능을 조금 포기하고 같은 GPU에 더 큰 모델을 올림. 손실 이유: 표현 가능한 값이 줄어 서로 다른 값이 같은 값으로 뭉침(0.7231, 0.7233, 0.7290 → 동일).
- 메모리 = 파라미터 수 × 값 하나의 바이트

형식	값 1개	7B	70B
FP32	4B	28GB	280GB
FP16/BF16	2B	14GB	140GB
INT8	1B	7GB	70GB
INT4	0.5B	3.5GB	35GB

큰 모델을 4bit로 > 작은 모델을 16bit로(대체로 더 나음). INT4 미만은 열화 뚜렷. 실제로는 KV 캐시 등이 더 붙음(7B INT4 ≈ 5GB). 방식: GGUF(CPU 포함, Ollama·LM Studio), AWQ/GPTQ(GPU, vLLM), BitsAndBytes(로딩 시 즉석, QLoRA).

성능 지표 · 모델 선택

지표	내용
MMLU	57개 주제 4지선다(지식·추론)
GPQA	대학원 수준, 구글링으로 안 풀리게 출제
HumanEval	코드를 실제 실행해 테스트 통과로 판정(pass@k)
SWE-bench	실제 GitHub 이슈+PR 2,294쌍(Verified는 500문항). FAIL→PASS(해결됐나), PASS→PASS(다른 걸 안 망가뜨렸나)

기존 지표는 새 모델이 나올수록 만점에 가까워져 새 지표가 계속 생김. 운영 지표: 컨텍스트 윈도우, 토큰당 가격, 지연시간(첫 토큰까지·초당 생성), 멀티모달, 라이선스.

	API 모델	오픈웨이트(로컬)
성능	최상위	근접하나 다소 뒤짐
비용	초기 없음, 토큰당 종량	GPU 서버 필요, 사용량 무관 고정비
데이터	외부로 전송	내부에 머묾
커스터마이징·버전	제한적, 모델 갱신 시 동작 변경 가능	미세조정 자유, 버전 고정

AI 시스템 구조

- 일반 웹 서비스에 오케스트레이션 · 검색 계층 · 모델 계층이 추가된 구조.

- 계층: 프론트엔드(React 등) → API 서버(인증·검증·로깅, FastAPI) → 오케스트레이션(검색·프롬프트 조립·LLM 호출 제어, LangChain) → 임베딩 / Vector DB / 모델 서빙(API·vLLM·Ollama) → 원본 저장소 → 운영(모니터링).
- 색인 시점: 문서 → 청킹 → 임베딩 → Vector DB 저장 / 질의 시점: 질문 → 임베딩 → 검색 → 프롬프트 조립 → LLM → 답변.
- Vector DB에는 벡터 + 원문 조각(답변 근거) + 메타데이터(권한 필터, 출처, 바뀐 조각만 재색인) 저장. 파이썬이 표준인 이유: 임베딩·LLM SDK·벡터 DB 클라이언트가 모두 파이썬 우선.
- 실무 혼합 구성: 임베딩은 로컬 소형(호출량 많음), 생성은 API(품질). 지연은 대부분 LLM 첫 토큰까지에서 발생.

REST API · FastAPI

- REST의 핵심 두 가지 = 메서드 + 자원(URI). URL은 전체 주소, URI는 그중 자원을 식별하는 `/` 이후 경로. `{id}`는 단건(키).

메서드	의미	안전(상태 안 바꿈)	멱등(여러 번 해도 같음)
GET	조회	O	O
POST	생성	X	X
PUT	전체 교체	X	O
PATCH	부분 수정	X	X
DELETE	삭제	X	O

- URI는 복수형 명사, 동사는 메서드가 담당. 나쁜 예 `POST /createUser` → 좋은 예 `POST /users`, `DELETE /users/1`. 계층은 경로로(`/users/1/documents`), 소문자·하이픈, 확장자 없음.

설계 연습: documents CRUD	경로	동작	응답
GET	/documents	목록 조회	200
GET	/documents/{id}	단건 조회	200 / 404
POST	/documents	생성(업로드)	201
PATCH	/documents/{id}	부분 수정	200
DELETE	/documents/{id}	삭제	204

상태 코드: 2xx 성공(200, 201, 204) · 3xx 리다이렉트 · 4xx 클라이언트 잘못(400, 401, 403, 404, 422, 429) · 5xx 서버 잘못(500). AI 시스템 예: `POST /api/chat`(스트리밍), `POST /api/documents`(색인은 오래 걸리니 즉시 반환 후 백그라운드), `POST /api/feedback`, `GET /health`.

- FastAPI 3요소: Starlette(라우터: 메서드+URI를 보고 어떤 함수를 호출할지 정함, ASGI) · Pydantic(데이터 검증·문서 자동 생성) · Uvicorn(요청을 받아 앱에 전달하는 서버). 요청 → Uvicorn → Starlette → Pydantic 검증.
- WSGI(Flask 등)는 요청 하나가 끝날 때까지 워커 점유, ASGI(FastAPI)는 대기 중인 요청을 내려놓고 다른 요청 처리.

> 비동기가 중요한 이유: LLM 호출은 계산(수 ms)이 아니라 네트워크 대기(2~10초)가 대부분. 동기면 대기 동안 워커가 묶여 처리량 급감. 처리량 = 워커 수 ÷ 평균 응답 시간 (워커 4, 응답 5초 → 0.8건/초). 비동기는 `await`에서 작업을 대기 목록에 넣고 다른 요청을 처리하다가 응답이 오면 이어서 재개 → 워커 1개로도 수백 건 동시 대기. 워커를 늘리는 건 워커당 메모리 수백 MB라 비현실적.

- Pydantic: 잘못된 요청은 엔드포인트 진입 전에 422로 차단, 오류 필드 메시지 자동, `/docs` 자동 문서화. `max_length` 없으면 10만 자 입력 → 토큰 비용 사고. `top_k` 상한, 모델명은 `Literal` 허용값, 응답 스키마 고정(sources 빠지면 근거 표시 깨짐).
- 구조화 출력: 텍스트가 아니라 미리 정한 형식(JSON)으로 출력. 코드가 출력을 바로 쓸 때(에이전트가 명령어를 내뱉을 때) 필요. LLM은 형식을 어길 수 있어 검증 + 재시도가 기본 패턴.
- 스트리밍: 8초 빈 화면 대신 0.8초에 첫 글자. 총 시간은 같아도 체감이 다름. TTFT(첫 토큰까지), TPS(초당 토큰).

3주차 · 프롬프트 · 컨텍스트 엔지니어링

엔지니어링이 등장한 3가지 배경

배경	내용 → 결론
① 모델을 학습시킬 수 없음	사전학습 불가, 미세조정 제한적 → 모델 주변(프롬프트·컨텍스트)을 설계. 가장 쉬운 프롬프트부터 시작
② 출력이 확률적(non-deterministic)	같은 입력도 매번 다름(temperature 0이어도 완전히 같진 않음). 통과/실패 대신 지표(근거 충실, 질문에 답했나, 형식 준수, 출처)로 측정. → 출력 형식 강제·검증, 실패 시 재시도 경로, 품질 반복 측정
③ 컨텍스트가 유한하고 비쌈	시스템 지시+도구 명세+이력+검색 결과+질문이 합산, 길수록 비용·지연 ↑ (어텐션 $O(n^2)$), 많이 넣는다고 좋아지지 않음(불필요한 정보는 간섭, 가운데 정보는 반영이 약함) → 무엇을 넣을지 고르는 것이 성능이자 비용

등장 순서: 프롬프트 → 컨텍스트 → 하네스 → 루프 (뒤의 것이 앞의 것을 포함). "요령(문구 복사)"에서 "공학(실패 유형 분류·버전 관리·지표 측정·계층별 책임 분리)"으로. 프롬프트는 일반 사용자도 가능, 컨텍스트부터는 개발자 영역.

LLM 엔지니어링은 로그 대신 증상을 보고 어느 계층이 원인인지 판단해야 함 → 증상별 원인 계층과 대응을 함께 정리해 둘 것.

프롬프트 엔지니어링

- 구성: 시스템 프롬프트(역할·규칙, 사용자에게 안 보임), 사고 유도(Chain-of-Thought), 구조화 출력. 같은 모델도 "역할 + 사내 규정 + 출력 형식 + 근거 요구"를 주면 결과가 완전히 달라짐.
- 구조화 출력은 코드가 출력을 바로 써야 할 때 사용(에이전트가 도구 호출 명령을 내뱉을 때). 무조건 좋은 게 아니라 언제 쓰는지를 이해할 것.

컨텍스트 엔지니어링

- 프롬프트 = 질문을 잘 하는 것, 컨텍스트 = 모델이 한 번 호출할 때 보는 것 전부를 설계: 시스템 프롬프트 + 대화 이력 + 검색 결과 + 도구 명세 + 사용자 질문. 어디에 배치하고 토큰을 얼마나 줄지도 설계.
- Lost in the middle: 정답 문서를 컨텍스트 앞/뒤에 두면 잘 쓰고, 가운데 두면 성능이 하락(실험에서 문서 없이 답할 때의 56.1%보다도 낮아짐). → 중요한 건 앞뒤에.

4가지 전략	핵심
선별(Select)	질문과 관련된 것만 골라 넣기 = RAG(상위 k개만)
압축(Compress)	길어지는 대화 이력 줄이기(아래 4방식)
격리(Isolate)	에이전트(서브에이전트)를 나눠 역할 분담 → 컨텍스트 윈도우가 여러 개. 단점: 수행이 오래 걸림
순서(Order)	가운데는 놓치므로 중요한 건 앞뒤에 배치

이력 압축 방식	방법	장점	단점
전체 유지	대화 전부 보관	단순, 정보 손실 없음	토큰이 바로 폭증(예: 1.8만)
슬라이딩 윈도우	최근 N턴만 유지	토큰 적음	이전 대화를 완전히 잊음
요약	최근 N턴 + 과거는 LLM이 만든 요약 1개	토큰 ↓, 맥락 일부 유지	요약 중 손실, 요약 호출 비용
선별 검색	전부 저장 후 현재와 관련된 것만 검색해 가져옴	관련 정보만 효율적	벡터 DB 등에 임베딩 저장 필요, 검색 성능에 의존

4주차 · 하네스 · 루프 엔지니어링

하네스(Harness) = 모델을 동작시키는 환경 설계 ("강아지 하네스")

모델의 가중치를 제외한 모든 것. 어떤 환경을 주고, 무엇을 하지 말아야 하는지 정함. 구성요소 6개:

구성요소	역할
도구 레지스트리	LLM의 손발. 쓸 수 있는 도구 목록. 프롬프트엔 코드가 아니라 도구 "설명"만 들어감 → 설명이 곧 프롬프트이므로 "무엇을 검색하는지" 구체적으로 써야 함
권한·승인	어떤 동작을 허락·확인받을지(위험 작업은 사용자 승인)
환경 설정	어디에서만 작업할지(범위 제한)
컨텍스트 조립	컨텍스트 엔지니어링의 구성요소를 모아 조립
검증 게이트(가장 중요)	출력 형식이 맞는지, 나가면 안 되는 내용이 없는지, 실제로 실행했을 때 통과하는지 검증
상태·관측	어디까지 수행했는지 기록. 루프 엔지니어링을 위해 필요

에이전트 종류: 라우터, 플래너, 가드레일(보안) = 방화벽처럼 모델 입력과 출력을 모두 걸러줌.

> 도구 호출이 뒤에서 동작하는 방식 (사용자: "오늘 서울 날씨 알려줘")
> ① 도구 목록을 JSON 스키마로 만들어 에이전트 LLM에 전달 → ② LLM이 질문과 도구 설명을 보고 `get_weather` 가 필요하다고 판단, 파라미터 `city` 가 질문에 있으니 "서울"로 채움 → ③ LLM이 명령어(구조화 출력)를 내뱉음(텍스트인데 그 텍스트가 명령어) → ④ 백엔드가 그 명령어를 해석해 실제 함수를 실행 → ⑤ 결과를 LLM에 다시 전달 → 최종 답변.
> ※ LLM이 직접 실행하는 것이 아님. 구조화 출력이 이 과정의 기반.

- 도구가 수백 개일 때: 설명을 전부 프롬프트에 넣으면 토큰을 다 잡아먹음 → ① 도구 설명을 임베딩해 벡터 DB에 넣고 질문과 유사한 top-k 도구만 전달 ② 라우터 모델(경량 분류 모델)로 후보를 줄임, 선별된 스키마를 LLM에 주어 호출 여부까지 결정.

루프(Loop) 엔지니어링

- 목표를 정해 두고 달성할 때까지 LLM을 반복 호출. 기본 4단계: 관찰 → 행동 → 검증 → 복구(재시도). 관찰은 하네스의 상태·관측 기록을 이용, 검증에서 실패하면 복구하고 다시 처음으로 돌아가 루프.
- 가장 중요한 설계 = 종료 조건(언제 끝낼까). 종료 조건을 잘못 정해서 생기는 현상들이 안티패턴(무한 반복·비용 폭증 등) → 증상별 대응(최대 반복 횟수·토큰/비용 상한 등)을 이해할 것.
- 하네스·루프가 필요한 경우 / 불필요한 경우를 구분(단순 질의응답에는 과함, 여러 단계의 작업·검증이 필요한 에이전트형 작업에는 필요).
- Agentic RAG는 이 루프 안에서 사용되는 RAG.

5주차 · RAG · 임베딩 · 벡터 DB · 청킹 · 검색 품질

RAG 개요

- Retrieval-Augmented Generation: 질의 시점에 문서를 검색해 근거로 넣고 생성. 가장 많이 쓰는 저장소 = 벡터 DB. 두 시점: 인덱싱 시점(데이터를 쌓음) / 질의 시점(검색해 가져옴).
- 장점: 환각 감소(가장 큼) · 출처 제시 가능 · 사내 규정·최신 자료는 학습으로는 매일 학습해야 해서 불가능한데 RAG가 해결 · 갱신이 쉬움(벡터 DB만 갱신) · 관련 조각만 넣으므로 비용 효율.
- 단점: 검색이 실패하면 끝(검색 성능에 의존) · 여러 문서를 가로지르는 질문이 어려움(조각 간 관계 부족) · 근거가 있어도 무시(컨텍스트 가운데 문제) · 근거 자체가 틀린 경우(웹 검색 등) → 출처가 중요.

> 임베딩 모델 규칙
> · 색인할 때와 질문할 때의 임베딩 모델은 반드시 같아야 함(같은 벡터 공간에서 비교해야 하므로).
> · 반대로 벡터 DB의 임베딩 모델과 LLM의 임베딩 모델은 달라도 됨: 검색 결과로 LLM에 넘어가는 것은 벡터가 아니라 원문 텍스트이기 때문.

RAG 종류 — "무엇을 바꾸는가"로 3분류

분류	종류	핵심
① 데이터(인덱싱) 구조를 바꿈	GraphRAG	조각을 노드로, 조각 간 관계를 엣지로(관계는 문장 요약). 인덱싱할 때 에이전트가 관계를 자동 생성 → 비용·데이터량 큼. 문서 간 연결 질문에 강함
② 검색 방법을 바꿈	하이브리드 검색	벡터 + BM25를 RRF로 합침
	리랭킹 / 질문 재작성 / HyDE	아래 "검색 품질" 참고
	멀티모달	텍스트가 아닌 것도 데이터로 넣을 수 있음
③ 판단·검증 반복을 넣음	Self-RAG	모델이 4가지 반성 토큰을 내뱉도록 미세조정(학습). 다른 RAG와 달리 학습이 들어감. 생성 모델이 스스로 판단
	CRAG(Corrective)	검색 결과를 판단하는 별도 모델(에이전트)이 있어 쓸지 말지 결정. Self-RAG와 차이: 판단자가 따로 있음
	Adaptive RAG	질문이 들어오면 검색을 할지 말지를 먼저 판단. 간단한 질문은 검색 없이 답 → 토큰 절약
	Agentic RAG	에이전트를 검색 루프에 넣음. 결과가 나쁘면 다시 검색, 종료 조건까지 반복 (루프 엔지니어링 안에서 사용)

임베딩

- 문장 → 고정 길이 벡터(예: 768차원). 뜻이 비슷하면 가까움. 정규화되어 있으면 내적 = 코사인 유사도.
- 한계: 부정문("할 수 있다/없다") 구분 약함, 코드·숫자·이름(KCC-4/KCC-5)은 의미상 비슷해 벡터가 거의 같음, 동의어는 오히려 강점.
- 모델 선택 기준: 차원 수(메모리·속도), 언어(한국어 지원), 비용. 예) 메모리가 4GB밖에 없으면 차원이 작은 임베딩 모델 선택.

벡터 DB 인덱스 (ANN)

- Flat(정확 탐색, 브루트포스): 질문과 모든 벡터를 비교. 항상 정확하고 구현 쉽고 삽입·삭제 자유. 비용 = 조각 수 × 차원 수(선형 증가). 규모가 크지 않으면(예: 과제의 조각 60개) 인덱스는 의미 없고 Flat이 최선 (GPU 행렬곱이 빠름).
- B+트리를 못 쓰는 이유: RDBMS는 키를 정렬해 트리로 찾는데, 고차원 벡터는 정렬이 불가능("이 점보다 작은 점"이 어느 방향인지 모름).
- ANN(근사 최근접 이웃): 빠르게 찾는 대신 정답을 놓칠 수 있음. 평가: Recall@k = 정확 탐색의 상위 k개 중 인덱스로 찾은 비율(10개 중 9개 → 0.9). 보통 0.90~0.95로 맞춤(1.0이면 거의 다 본 것이라 속도 이점이 없음).

- 전략 3가지: 구역 나누기(IVF), 그래프(HNSW) = 저장 구조를 바꿈 / 압축(PQ) = 비교 비용을 줄임 → 앞의 둘과 함께 사용(IVF-PQ, HNSW-PQ). 코드의 `IVFFlat`은 압축 없이 IVF만.

	IVF (Inverted File)	HNSW	PQ (Product Quantization)
원리	K-means로 `nlist`개 구역으로 나누고, 구역 중심(대표 벡터)과 먼저 비교해 가까운 `nprobe`개 구역만 탐색	층이 여러 개인 그래프. 맨 위층에서 시작해 가까운 이웃으로 이동, 더 가까운 이웃이 없으면 한 층 내려감. 위층 노드는 아래층에도 존재, 위층 진입 확률은 낮아 맨 아래층에 가장 많음	벡터를 m조각으로 나눠 조각별 K-means(K=256)를 돌리고, 원본 대신 대표 벡터 번호(1바이트)를 저장
파라미터	`nlist` ↑ → 구역이 작아 빠르지만 경계가 늘어 정답을 놓칠 확률↑. 시작값 = 조각 수의 제곱근(100만 → 1,000). `nprobe` ↑ → Recall↑ 속도↓. 시작값 = nlist의 1~5%. 질문마다 변경 가능(에이전트가 조절)	`M`(노드당 최대 연결 수)↑ → 촘촘, Recall↑ 메모리↑ 시간↑. `ef_construction`(구축 시 후보 수)↑ → 연결 품질↑ 구축 느림. 지역 최적해 방지용. `ef_search`(검색 시 후보 수)↑ → Recall↑ 속도↓. 질문마다 조절 가능	조각 수 m, K=256. 정보 손실 약간(양자화와 비슷)
특징·단점	K-means가 필요해 데이터가 어느 정도 쌓인 뒤에 구축. 분포가 크게 바뀌면 재구축	삽입 = 검색과 같은 과정 + 연결. 삭제는 그래프가 끊겨 "외딴섬"이 생김 → 삭제 표시만 해두고 나중에 재구축 → 삭제가 잦은 데이터에 불리. 데이터가 없는 상태에서도 시작 가능(콜드 스타트). 메모리 큼	질문 1개당 거리표(m × 256)를 한 번 만들고, 저장된 번호로 표를 조회해 덧셈만으로 거리 계산(곱셈 → 덧셈). 예: 1024차원 → 8조각 → 1024×4B=4096B를 8B로 = 512배 압축

벡터 DB

- 벡터 + 원문 조각 + 메타데이터(형식 자유)를 함께 저장하고 인덱스·검색 기능까지 묶은 시스템. 선택은 상황별: 필터 인식 검색이 중요 → Qdrant 등, 운영 부담 최소 → 관리형, 이미 PostgreSQL 사용 → pgvector(데이터 규모가 크지 않으면 가장 실용적) / 수천만 건 이상 → 전용 DB(Milvus 등).
- 필터(예: 인사팀 사용자는 인사팀 문서만): 후처리 필터 = 검색을 끝낸 뒤 메타데이터로 거름 → 상위 5개 중 조건에 맞는 것만 남아 k개를 못 채울 수 있음. 필터 인식 검색 = 탐색 중에 조건을 걸어 항상 k개 반환(Qdrant 등이 제공). 권한 문제에서 중요.

청킹

- 왜 자르나: 컨텍스트 윈도우가 한정적 → 필요한 정보만 가져오려고. 문제 3가지: ① 맥락 단절 ② "이 경우…" 같은 대명사의 지칭 대상이 앞 조각에 있어 알 수 없음 ③ 경계: 정답이 두 조각에 걸침.

방식	방법	특징
고정 길이	단어·문장 상관없이 N자마다	가장 쉬움(코드 한 줄), 문장이 잘림 → 오버랩 사용
재귀 분할	구분자 목록(문단 → 줄 → 문장 → 단어)과 청크 크기를 정해 두고, 크기 이하가 될 때까지 다음 구분자로 재귀, 너무 작은 조각은 합침	크기 보장, 빠르고 쌈, 문서 형식을 몰라도 동작, 구조(문단·줄 경계) 존중
구조 기반	제목/조항(`## 제N조`, 마크다운 헤딩)으로 자름	규정·법률·MD에 적합. 문서 구조를 미리 알아야 함(단점)
의미 기반	문장 단위로 쪼개 임베딩 → 인접 문장 간 의미가 크게 튀는 지점을 경계로	임베딩 모델 성능이 중요, 임계값 설정 필요, 느리고 비쌈

- 오버랩: 경계에서 잘린 문장을 앞뒤 한쪽에 온전히 남기려고 겹침(대신 조각 수·저장량 증가). 청킹 방식에 따라 필요 여부가 달라짐(고정 길이는 필요, 구조 기반은 조 단위로 자르므로 보통 불필요) → 왜 쓰는지/안 쓰는지 설명할 수 있어야 함. 문서 유형(회의록·QnA·웹 문서·규정)별 청킹 선택도 고민할 것.

- 과제 결과(Recall@5): 500자 고정 0.65 / 문장 단위 0.50(주제어 없는 조각 생성) / 조 단위 구조 기반 0.85 / + 하이브리드(BM25+RRF) 0.90.

검색 품질

- 검색 실패 3유형: 못 찾음(후보에 없음) / 잘못 고름(순위 낮음) / 찾았는데 안 씀(LLM이 무시). 벡터 검색은 사람 이름·부서명·코드(KCC-3 vs 5)·숫자에 약함 → BM25를 섞음.
- BM25: 단어가 얼마나 드문지(IDF: 그 단어가 들어간 조각 수 n_q 기준, 드물수록 큼) × 조각에서 얼마나 자주 나오는지(TF, 횟수가 늘어도 상한선이 있음 — 너무 많으면 한 조각이 우월해지므로) ÷ 길이 보정(긴 조각은 키워드가 우연히 포함될 확률이 높으므로 분모에서 보정). 뜻 대신 글자 일치.

> 하이브리드 검색 + RRF
> 질문 → 벡터 검색(코사인) 상위 50 + BM25(희소) 상위 50 → RRF로 합침. 두 점수는 단위가 달라(코사인 0~1, BM25 상한 없음) 정규화가 안 되므로 순위만 사용.
> RRF 점수 = Σ 1 / (k + 순위), k = 60. 예) 문서 A: 벡터 1위, BM25 2위 → 1/61 + 1/62. 순위가 높을수록 점수가 높고, 양쪽에서 모두 상위권이면 최고점.

기법	방법	언제 / 단점
리랭킹	1차로 넓게(예: 100개) 가져온 뒤 교차 인코더로 질문+조각을 한 입력으로 붙여 어텐션 → 관련도 점수 1개(스칼라) → 상위 5개 선택	더 정밀. 딥러닝 모델을 한 번 더 쓰므로 느려짐 → 기다림을 허용하는 시스템이면 사용. (일반 임베딩은 질문과 조각을 따로 벡터화해 코사인 비교)
질문 재작성	애매한 질문을 구체적으로: "그거 언제야?" → "2026학년도 2학기 수강신청 시작일은?" (에이전트가 앞뒤 문맥 보고 재작성)	애매모호한 질문이 많을 때
다중 질의 (Multi-query)	질문을 여러 관점으로 늘려(수강신청 일정? 등록 기간? 학사 일정?) 각각 검색 → RRF로 합침	다양한 관점의 답변 가능
HyDE	질문에 대한 가상 답변을 먼저 생성하고 그 벡터로 검색	짧은 질문과 긴 문서의 길이·형식 괴리를 줄임. 가상 답변이 부정확해도 형식은 비슷하므로 검색이 수월

지표	의미
Recall@k	정답 조각이 상위 k개 안에 들어왔는지(과제 지표)
Precision@k	상위 k개 중 실제 관련 있는 비율 (5개 중 3개 → 3/5)
MRR	첫 정답 순위의 역수(1/순위)의 평균
Hit Rate	상위 k개에 정답이 하나라도 있는 질문의 비율

6주차 · LangChain · LangGraph

> 이 단원은 아직 슬라이드와 한 줄씩 대조하지 못한 요약본입니다. 교수님 힌트: 코드를 외우지 말고 "구성요소를 체인으로 엮는 방식"을 이해할 것. 특히 앞 구성요소의 출력 형태가 다음 구성요소의 입력 형태와 맞아야 체인이 된다("무조건 갖다 붙인다고 체인이 되는 게 아님").

- LangChain = 오케스트레이션 프레임워크. 프레임워크 없이 RAG를 만들 때 직접 짜야 하는 흐름(질문 → 검색 → 프롬프트 조립 → LLM → 출력)을 구성요소(프롬프트 · 모델 · 출력 파서 · 리트리버 등)를 가져와 체인으로 표현.
- LCEL: 구성요소를 파이프(`|`)로 연결. 각 구성요소는 Runnable이라 `invoke`(1건), `stream`(토큰 스트리밍), `batch`(여러 건), 비동기 버전을 공통으로 지원. 예) 프롬프트(딕셔너리 → 메시지) → 모델(메시지 → AI 메시지) → 파서(AI 메시지 → 문자열).
- `RunnableParallel`(여러 가지를 동시에 실행해 딕셔너리로 모음 — 예: 검색 결과와 질문을 함께 프롬프트에 전달), `with_retry`(실패 시 재시도), `with_fallbacks`(실패 시 대체 모델/경로).
- LangGraph: 노드·엣지로 흐름을 그래프로 표현해 분기·반복(루프)이 필요한 에이전트형 흐름에 사용.

서술형 대비 · 인과관계 한 줄 모음

질문	한 줄 답
글을 2배로 늘리면 왜 비용이 4배가 되나?	어텐션이 모든 토큰 쌍을 비교해 계산량이 n^2이므로.
왜 문서를 통째로 넣지 않고 RAG를 쓰나?	컨텍스트 윈도우 상한 + 길수록 비용·지연 ↑ + 불필요한 정보는 간섭·가운데 정보는 무시되므로 필요한 조각만 골라 넣는 것이 성능이자 비용.
사전학습만 한 모델이 질문에 답하지 못하는 이유?	다음 토큰 예측만 학습해 이어 쓰기만 하기 때문. SFT가 응답 형식을 가르침.
양자화하면 왜 성능이 떨어지나?	표현 가능한 값이 줄어 서로 다른 값들이 같은 값으로 뭉치므로.
한국어는 왜 토큰이 많이 드나?	영어 데이터 비중이 커 병합 기회가 적고, 조사·어미 때문에 같은 어간이 여러 형태로 흩어져 빈도가 분산되므로.
FastAPI(비동기)가 LLM 서버에 적합한 이유?	LLM 호출은 대부분 응답 대기 시간이라, 대기 중 워커를 놓아 다른 요청을 처리하면 동시 처리량이 크게 늘기 때문.
URI에 동사를 넣지 않는 이유?	동작은 메서드(GET/POST/DELETE…)가 표현하고 URI는 자원(명사)만 식별하게 해 일관된 설계를 만들기 위해.
구조화 출력은 언제 필요한가?	LLM 출력을 사람이 아니라 코드가 바로 사용할 때(에이전트의 도구 호출 명령 등).
하네스에서 검증 게이트가 왜 중요한가?	LLM 출력은 확률적이라 형식·금지 내용·실행 통과 여부를 확인하지 않으면 틀린 결과가 그대로 나가고, 루프의 복구 판단 근거도 없어지므로.
루프에서 종료 조건이 가장 중요한 이유?	종료 조건이 없거나 잘못되면 무한 반복·비용 폭증 같은 안티패턴이 생기므로.
중요한 정보는 컨텍스트의 어디에 두나?	앞이나 뒤. 가운데는 모델이 잘 반영하지 못해 성능이 떨어지므로.
색인과 질문의 임베딩 모델이 같아야 하는 이유?	서로 다른 모델은 벡터 공간이 달라 유사도 비교가 의미가 없으므로. (LLM 쪽 모델과는 달라도 됨 — 넘어가는 건 벡터가 아니라 원문 텍스트.)
벡터 검색만으로 부족한 이유 / 해결?	이름·코드·숫자(KCC-3 vs 5)는 의미가 비슷해 벡터가 가까워 구분이 안 됨 → BM25(글자 일치)를 섞어 하이브리드 + RRF.
RRF에서 점수 대신 순위를 쓰는 이유?	코사인(0~1)과 BM25(상한 없음)의 단위가 달라 점수를 그대로 합칠 수 없으므로.
리랭킹의 장단점?	질문+조각을 함께 어텐션해 더 정밀하게 재정렬하지만 모델을 한 번 더 돌려 느려짐.
IVF에서 nlist를 늘리면?	구역이 작아져 빠르지만 경계가 늘어 정답을 놓칠 확률이 커짐. nprobe를 늘리면 Recall ↑ 속도 ↓ .
HNSW가 삭제에 불리한 이유?	삭제하면 그래프 연결이 끊겨 도달 불가능한 외딴 노드가 생기므로 삭제 표시만 하고 재구축해야 함.
PQ가 검색을 빠르게 만드는 이유?	벡터를 대표 번호로 압축하고 질문별 거리표를 미리 만들어 거리 계산을 표 조회와 덧셈으로 바꾸므로.
후처리 필터의 문제?	검색 후에 거르면 상위 k개 중 조건에 맞는 것만 남아 k개를 못 채움 → 필터 인식 검색은 탐색 중에 조건을 걸어 항상 k개.
Self-RAG와 CRAG의 차이?	Self-RAG는 생성 모델 자체를 미세조정해(학습) 스스로 판단, CRAG는 검색 결과를 판단하는 별도 모델이 있음.
정답이 후보에는 있는데 5위 밖일 때?	순위가 낮은 유형 → 리랭킹으로 재정렬(청킹 문제가 아님).

계산 모음

유형	공식 / 예
모델 메모리	파라미터 수 × 바이트. 13B FP16 = 26GB, 70B INT4 = 35GB, 7B INT8 = 7GB
처리량(동기)	워커 수 ÷ 응답 시간. 4 ÷ 5초 = 0.8건/초
어텐션	n배 길이 → n^2배 계산 (2배 → 4배)
RRF	Σ 1/(60+순위). 양쪽 1위 = 1/61 + 1/61 ≈ 0.0328
IVF 탐색량	100만 개, nlist 1,000, nprobe 10 → 중심점 1,000개 + 10구역 × 1,000개 ≈ 11,000회(전체의 약 1%)
PQ 압축률	(차원 × 4B) ÷ (조각 수 × 1B). 1024차원, 8조각 → 4096 ÷ 8 = 512배
Precision@k / Recall@k(ANN)	5개 중 3개 관련 = 3/5 / 정확 탐색 상위 10개 중 9개 찾음 = 0.9
BPE 어휘	초기 글자 수 + 병합 횟수